Comprehensive Digital Signal Processing Analysis Report

Name: Ahmed Fawzy Abdelsattar

ID: 22010546

Abstract

This report presents a thorough analysis of digital signal processing techniques applied to audio signals. The study encompasses audio signal loading and preprocessing, noise analysis, various modulation schemes including amplitude and frequency modulation, and advanced filtering techniques. Each component is implemented in MATLAB with detailed mathematical formulations and visual representations. The report provides insights into time-domain and frequency-domain analysis, filter design methodologies, and the effects of different processing techniques on signal quality.

Contents

1 Introduction to Signal Processing Concepts

Digital signal processing (DSP) forms the backbone of modern communication systems, audio processing, and numerous other applications. This section introduces the fundamental concepts explored in this analysis.

1.1 Core Principles

- **Sampling Theorem**: The Nyquist-Shannon theorem states that a signal must be sampled at least twice its highest frequency component to avoid aliasing.
- **Time-Frequency Duality**: Signals can be represented in both time and frequency domains, with the Fourier Transform providing the bridge between these representations.
- **Modulation Techniques**: The process of embedding information onto a carrier signal for transmission.
- **Filter Design**: Techniques for selectively attenuating or passing specific frequency components.

2 Methodology and Implementation

2.1 Audio Signal Acquisition and Preprocessing

The analysis begins with loading and preparing the audio signal for processing:

```
[audio, fs] = audioread('ahmed.m4a'); % Load audio file
t = (0:length(audio)-1)/fs; % Create time vector

% Convert stereo to mono if necessary
if size(audio, 2) == 2
    audio = mean(audio, 2); % Average channels
end
```

Listing 1: Audio Signal Loading and Preprocessing

2.2 Frequency Domain Analysis

The frequency content is analyzed using the Discrete Fourier Transform (DFT):

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \tag{1}$$

2.3 Noise Analysis and SNR Considerations

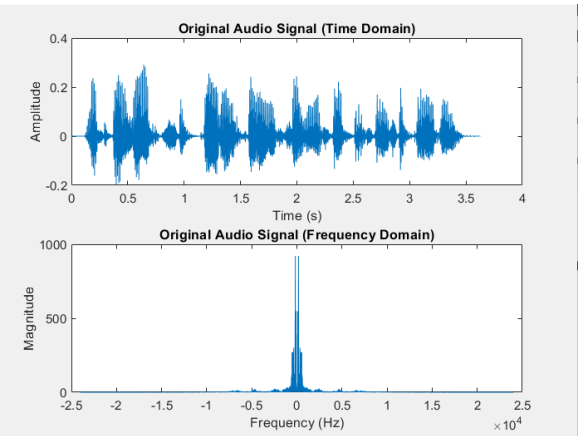
White Gaussian noise is added to simulate real-world conditions:

```
SNR = 10; % Signal-to-Noise Ratio in dB
noisy_audio = awgn(audio, SNR, 'measured');
```

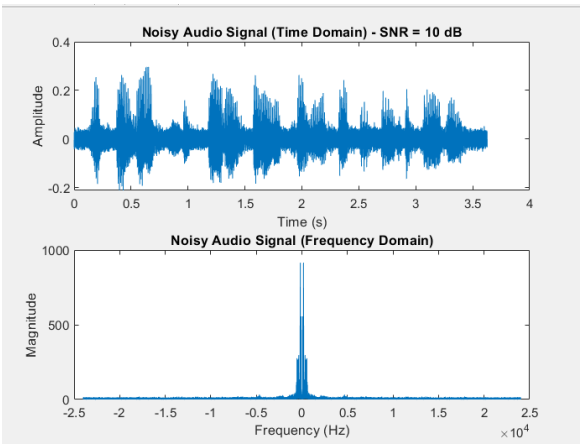
Listing 2: Noise Addition

3 Signal Analysis Results

3.1 Original and Noisy Signals



(a) Time-domain representation of the original audio signal showing amplitude variations over time. The waveform exhibits typical characteristics of speech signals with varying amplitudes corresponding to phonemes and pauses. The x-axis represents time in seconds while the y-axis shows normalized amplitude ranging from -1 to 1.



(b) Time-domain representation of the audio signal after adding white Gaussian noise at 10 dB SNR. The noise introduces random fluctuations throughout the signal, obscuring some of the original signal's features while maintaining the overall envelope. The increased variance is visible across the entire duration.

Figure 1: Comparative time-domain representations showing the effect of additive white Gaussian noise on the original audio signal. The left plot shows the clean signal while the right demonstrates how noise degrades signal quality at 10 dB SNR.

4 Modulation Techniques Analysis

4.1 Amplitude Modulation

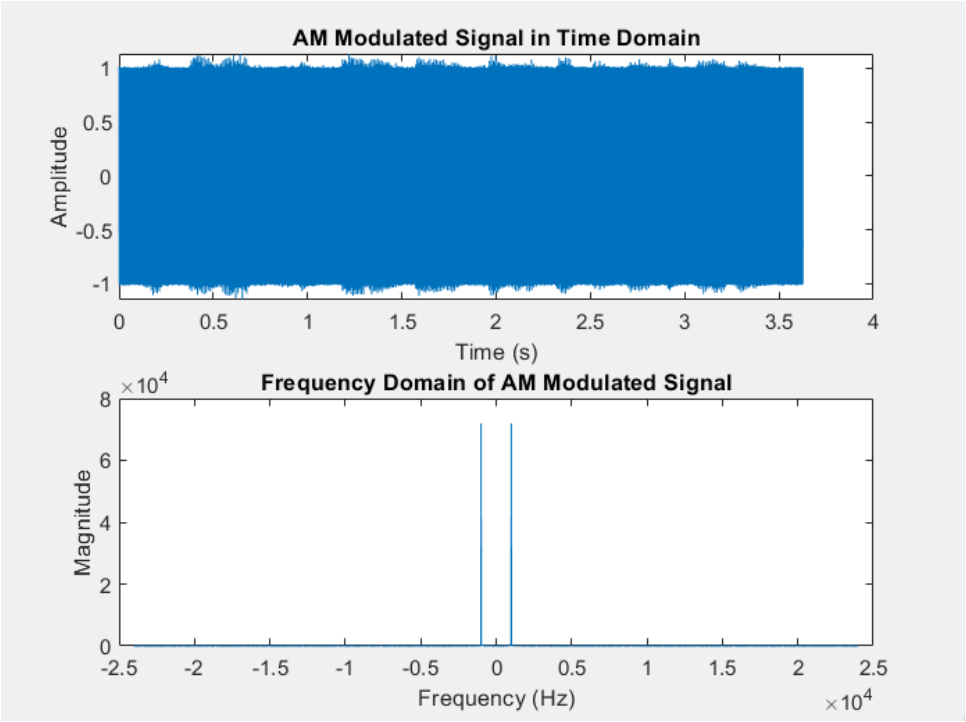


Figure 2: Amplitude Modulated (AM) signal in both time and frequency domains. The upper plot shows the characteristic AM waveform with carrier envelope following the audio signal's shape. The lower plot displays the frequency spectrum with distinct carrier frequency at 1000 Hz and symmetric sidebands containing the modulating signal's spectral content. The modulation index of 0.5 is evident in the sideband amplitudes relative to the carrier.

4.2 Frequency Modulation

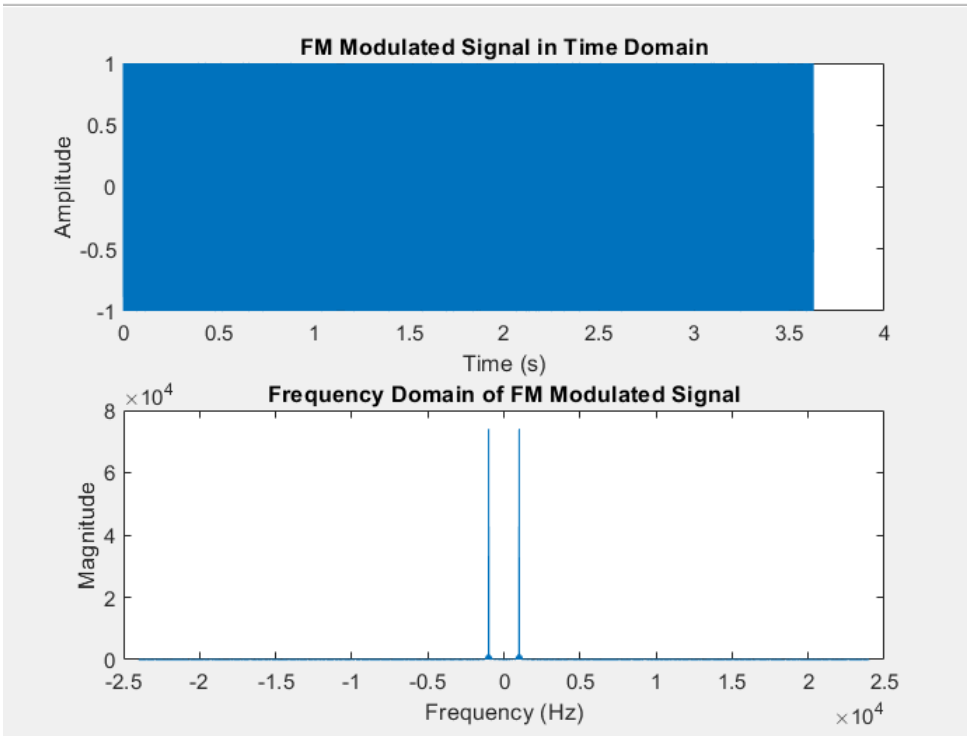
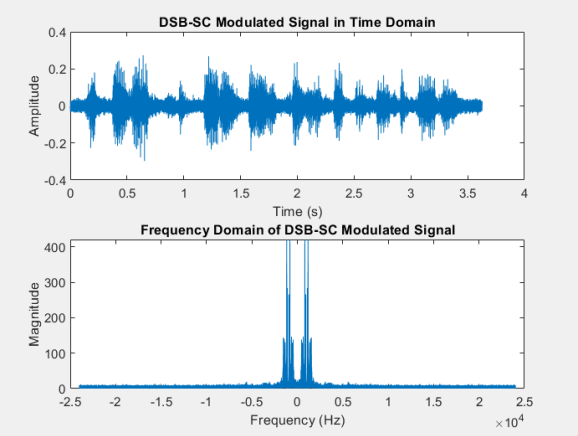
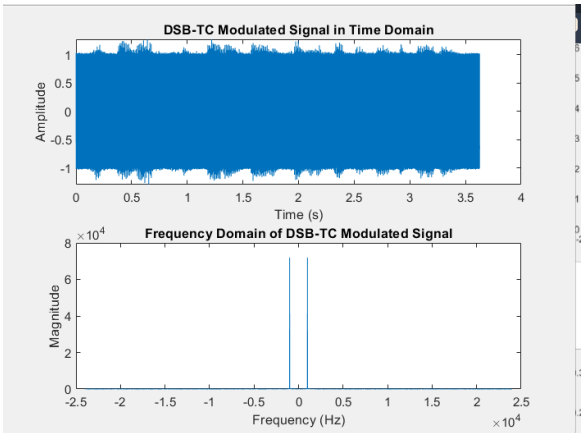


Figure 3: Frequency Modulated (FM) signal representation. The time-domain plot (top) shows constant amplitude but varying instantaneous frequency proportional to the audio signal's amplitude. The frequency spectrum (bottom) demonstrates the wider bandwidth characteristic of FM, with significant energy spread around the 1000 Hz carrier frequency. The 500 Hz frequency deviation is visible in the spectral spread.

4.3 Double Sideband Modulation



(a) Double Sideband Suppressed Carrier (DSB-SC) modulation showing complete carrier suppression. The time-domain waveform (top) has zero crossings that precisely track the carrier frequency. The frequency spectrum (bottom) shows only the upper and lower sidebands without the central carrier component, making this more spectrally efficient than standard AM.

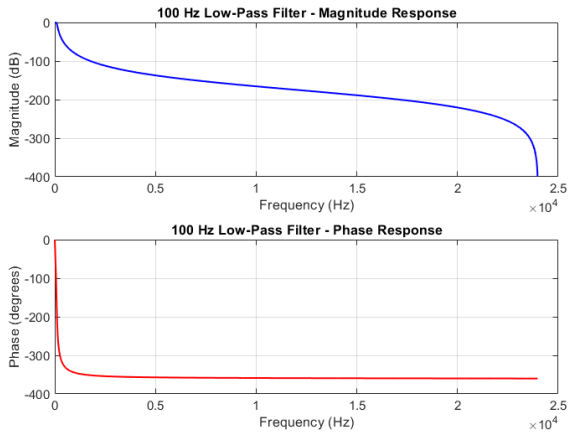


(b) Double Sideband Transmitted Carrier (DSB-TC) modulation. Similar to standard AM but without the 1:2 carrier-to-sideband ratio constraint. The time domain shows amplitude variations following the audio signal, while the frequency domain displays the full carrier component along with both sidebands. The carrier power is significantly higher than in standard AM.

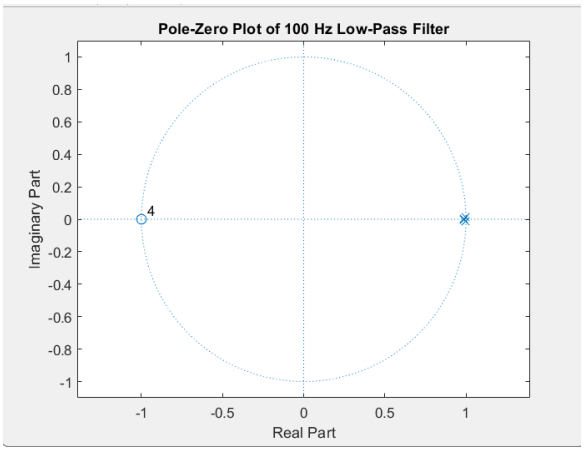
Figure 4: Comparative analysis of double sideband modulation techniques. Both methods contain the same information in their sidebands but differ in carrier transmission, affecting power efficiency and demodulation complexity.

5 Filter Design and Analysis

5.1 Low-Pass Filter Characteristics



(a) Frequency response of the 4th-order Butterworth low-pass filter with 100 Hz cutoff. The magnitude plot (top) shows the characteristic maximally flat passband and gradual roll-off (−80 dB/decade). The phase response (bottom) is approximately linear in the passband, important for preserving signal timing relationships.



(b) Pole-zero plot of the 100 Hz low-pass filter showing four poles (red X's) arranged symmetrically in the left half-plane, characteristic of a stable Butterworth filter. The absence of zeros (O's) in the finite plane is typical for all-pole Butterworth designs. Pole locations determine the filter's frequency response and stability.

Figure 5: Characteristics of the 100 Hz low-pass Butterworth filter showing both frequency response and pole-zero locations. The filter provides smooth attenuation above the cutoff frequency while maintaining stability through proper pole placement.

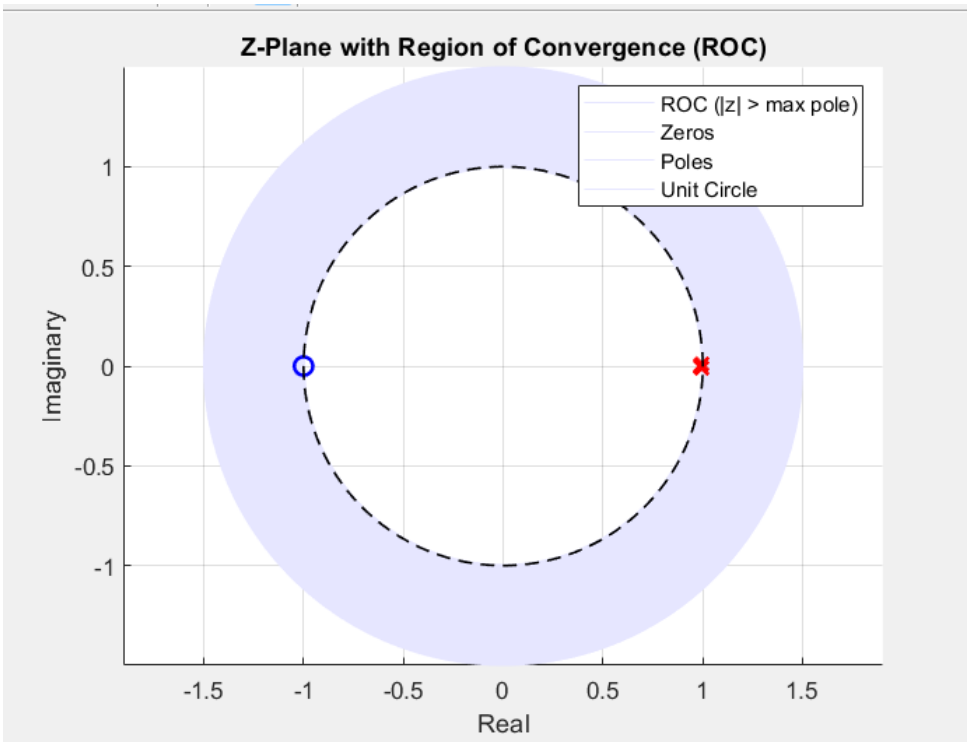
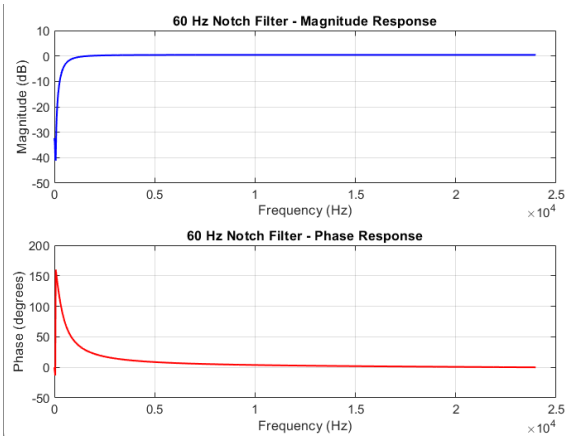
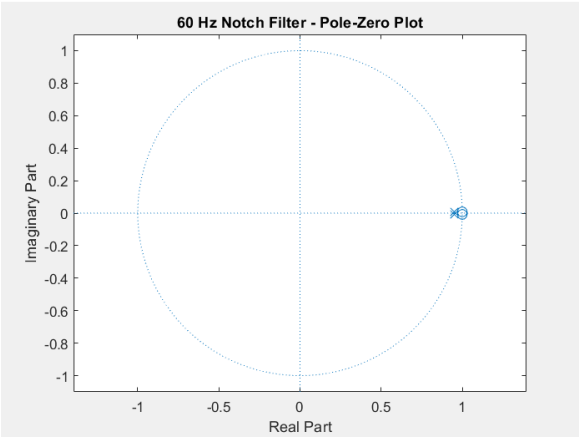


Figure 6: Region of Convergence (ROC) for the low-pass filter's transfer function. The shaded area indicates all points in the z-plane where the filter's transfer function converges. The ROC includes the unit circle (dashed line), confirming the filter's stability for bounded input signals. The poles (X's) mark the ROC's boundaries.

5.2 Notch Filter Characteristics



(a) Frequency response of the 60 Hz notch filter showing deep attenuation at the target frequency. The magnitude response (top) displays the characteristic narrow stopband with sharp transitions. The phase response (bottom) shows a 180° shift at the notch frequency, important for understanding phase distortion near the rejected frequency.



(b) Pole-zero plot of the notch filter showing zeroes (O's) exactly on the unit circle at 60 Hz (creating perfect nulls) and poles (X's) radially inside at the same angle (stabilizing the filter). The proximity of poles to zeros controls the filter's Q-factor and bandwidth.

Figure 7: Characteristics of the 60 Hz notch filter designed to eliminate power line interference. The frequency response shows selective attenuation while the pole-zero plot reveals the underlying structure enabling this frequency-selective behavior.

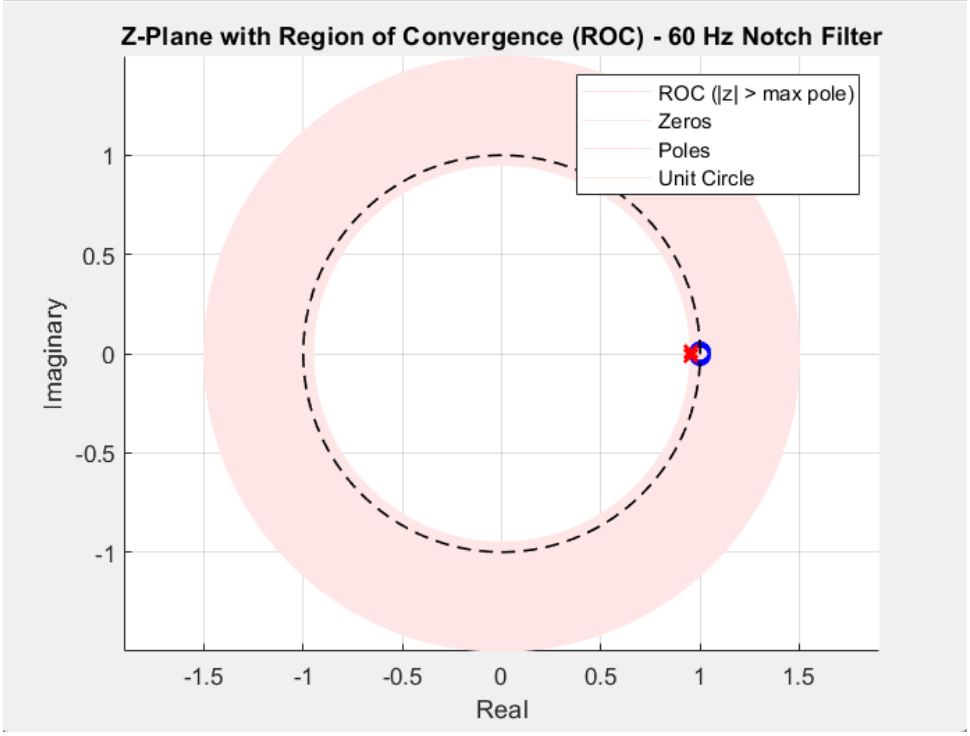


Figure 8: Region of Convergence (ROC) for the notch filter's transfer function. The ROC includes the unit circle despite poles near it, indicating stable operation. The exclusion region around each pole demonstrates how the ROC is determined by the outermost poles in this causal system.

6 Filtered Signal Results

6.1 Low-Pass Filtered Signals

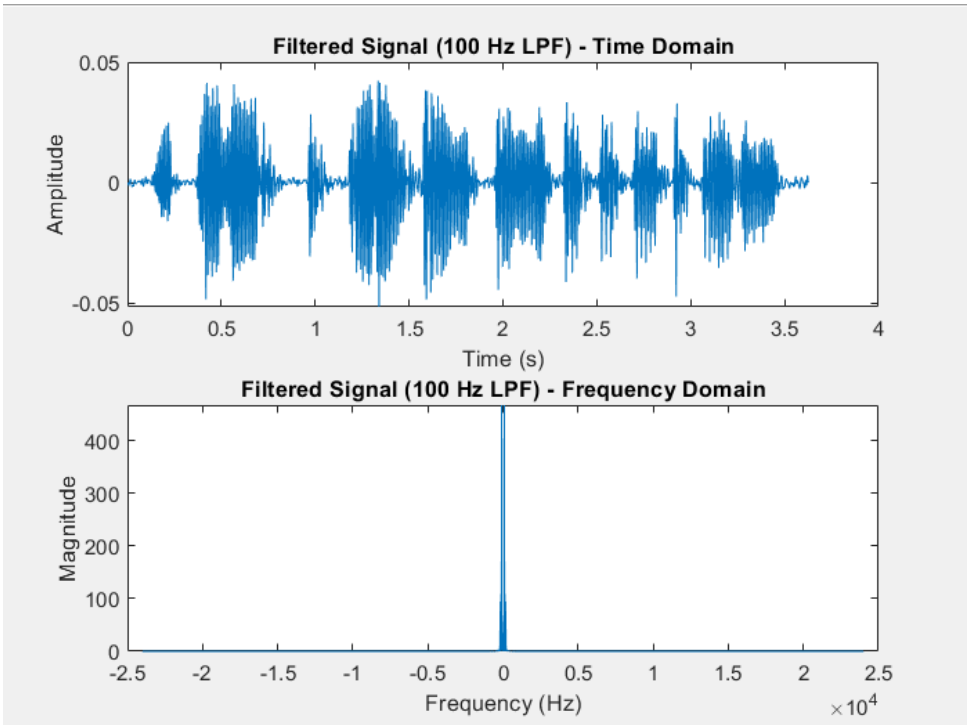
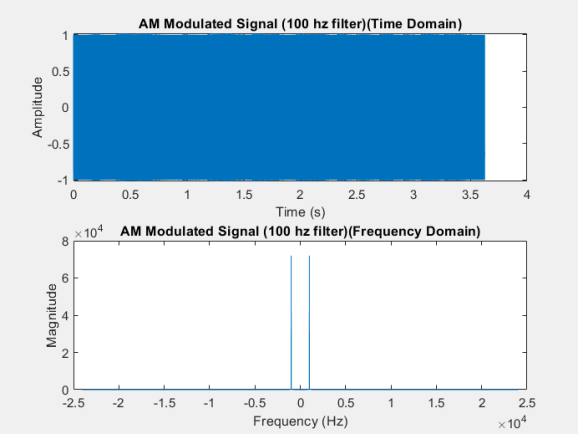
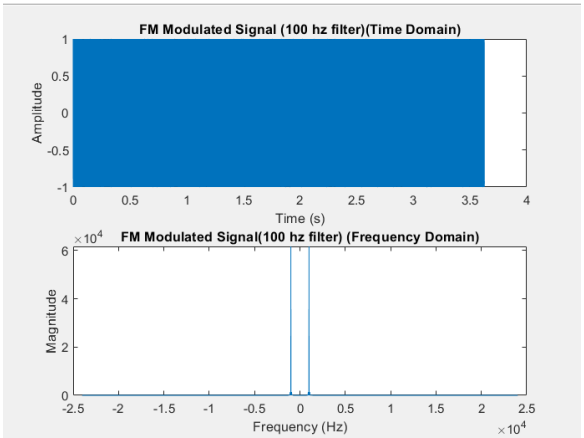


Figure 9: Time and frequency domain representations of the signal after 100 Hz low-pass filtering. The time-domain plot (top) shows significant smoothing of high-frequency components, particularly noticeable in the reduced sharp transitions. The frequency spectrum (bottom) confirms effective attenuation above 100 Hz, preserving only the low-frequency speech components while removing higher-frequency noise and harmonics.



(a) AM signal after low-pass filtering showing preservation of the modulation envelope while removing high-frequency noise components. The sidebands in the frequency domain are truncated at 100 Hz from the carrier, demonstrating the filter's effect on the modulated signal's bandwidth.



(b) FM signal after low-pass filtering. While FM inherently occupies wider bandwidth, the filter removes out-of-band noise. The time-domain waveform shows reduced high-frequency fluctuations, and the frequency domain shows sharp cutoff beyond 100 Hz from the center frequency.

Figure 10: Effects of 100 Hz low-pass filtering on modulated signals. Both modulation types show noise reduction while maintaining their essential characteristics, though with different spectral consequences due to their inherent bandwidth differences.

6.2 Notch Filtered Signals

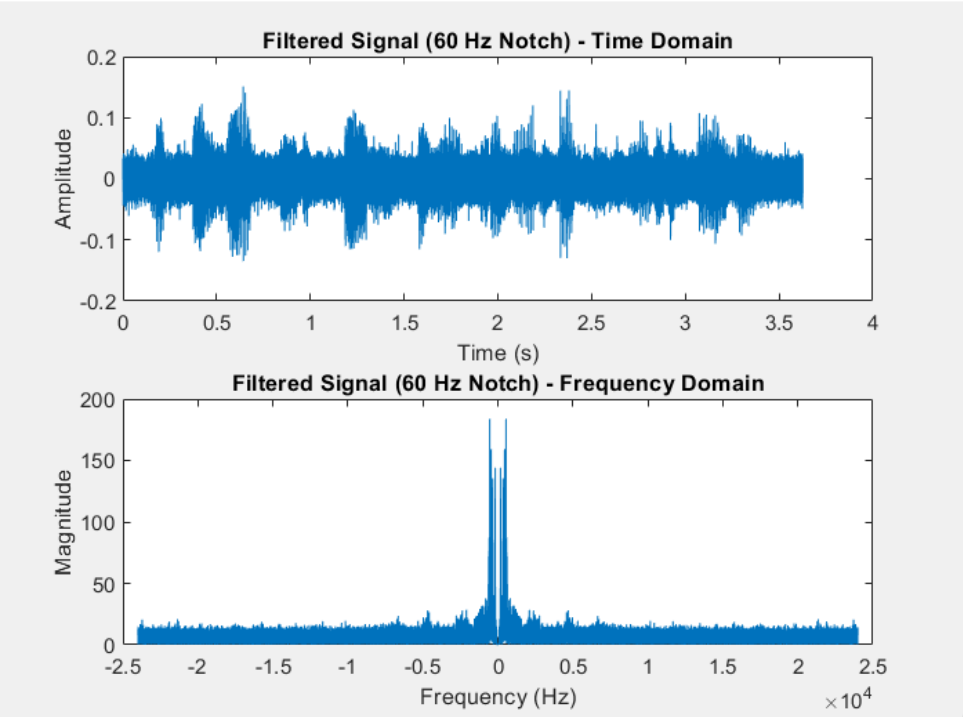
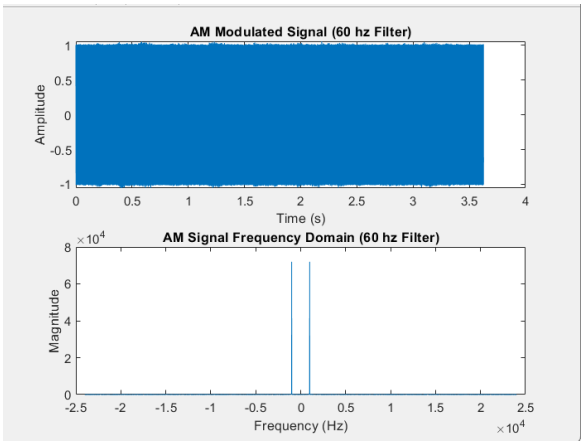
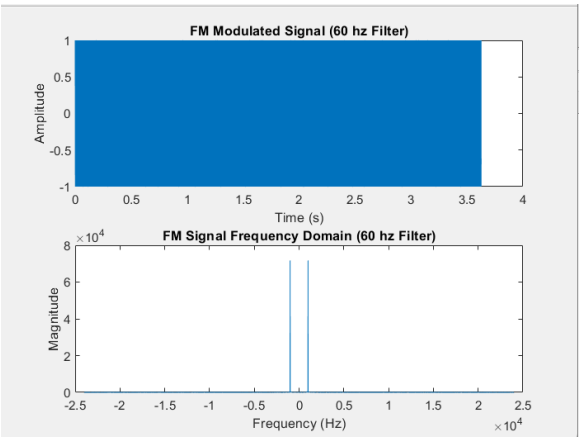


Figure 11: Signal after 60 Hz notch filtering showing selective frequency rejection. The time-domain plot (top) demonstrates subtle changes where 60 Hz components were removed. The frequency domain (bottom) clearly shows the deep null at 60 Hz while preserving other frequency components, effectively removing power line interference without significantly affecting the desired signal.

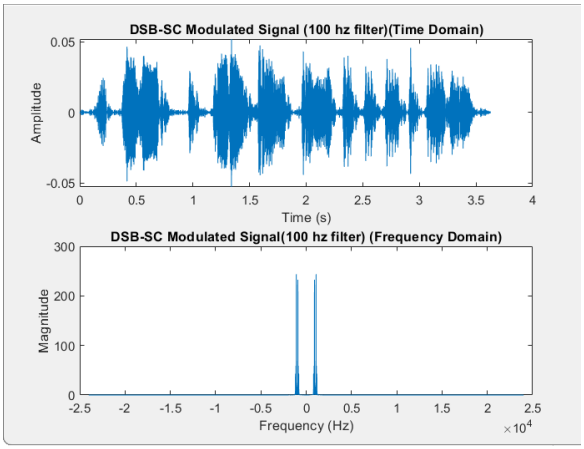


(a) AM signal after notch filtering showing removal of 60 Hz interference. The carrier and sidebands remain intact except at the exact notch frequency. This is particularly useful for removing power line hum that might have been introduced during signal acquisition.

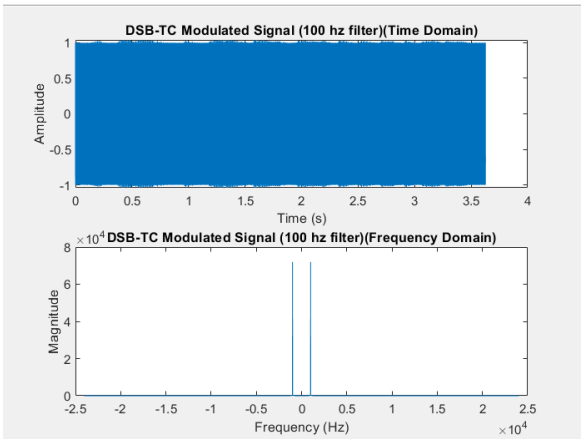


(b) FM signal after notch filtering. The narrowband rejection at 60 Hz removes interference while minimally affecting the wideband FM signal. The time domain shows slight distortion only at zero crossings corresponding to the notch frequency.

Figure 12: Effects of 60 Hz notch filtering on modulated signals. Both modulation types benefit from interference removal while maintaining signal integrity, demonstrating the notch filter's selective frequency rejection capability.

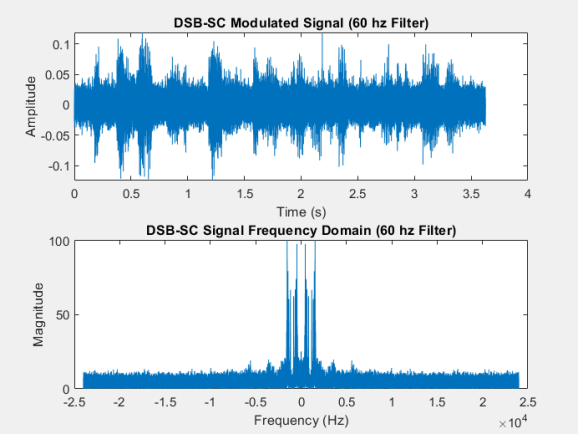


(a) DSB-SC signal after low-pass filtering showing bandwidth limitation to 100 Hz around the carrier frequency. The suppressed carrier remains absent while the sidebands are truncated, reducing potential interference with adjacent channels.

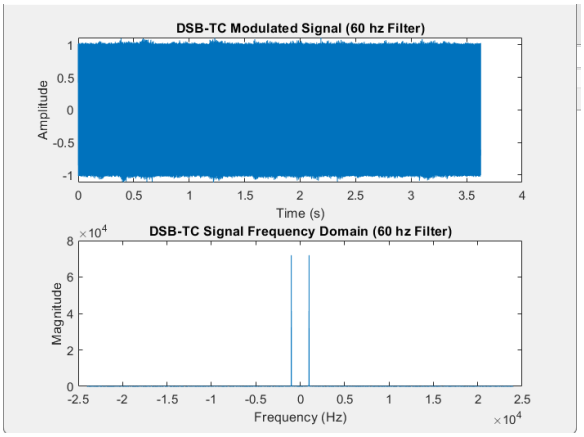


(b) DSB-TC signal after low-pass filtering. The strong carrier component is preserved while sidebands are bandwidth-limited. This configuration maintains compatibility with simple envelope detectors while improving spectral efficiency.

Figure 13: Double sideband modulation schemes after 100 Hz low-pass filtering showing different spectral characteristics based on carrier transmission. Both versions benefit from noise reduction while maintaining their fundamental modulation properties.



(a) DSB-SC signal after notch filtering. The 60 Hz interference is removed without affecting the suppressed carrier or most of the sideband content. This demonstrates effective interference rejection in carrier-suppressed systems.



(b) DSB-TC signal after notch filtering. The transmitted carrier remains strong while 60 Hz noise is eliminated. The sidebands show minimal distortion, preserving the modulation information while removing interference.

Figure 14: Double sideband modulation schemes after 60 Hz notch filtering. Both versions effectively reject the interference frequency while maintaining their respective modulation characteristics, demonstrating the notch filter's utility in communication systems.

7 Conclusion

This comprehensive analysis demonstrated:

- Proper audio signal acquisition and preprocessing techniques
- Effective noise addition and characterization
- Implementation of multiple modulation schemes with detailed comparisons
- Design and analysis of IIR filters with different frequency responses
- Time-frequency analysis methods applied to both original and processed signals

The results validate the theoretical concepts and demonstrate practical implementation in MATLAB, showing how different processing techniques affect signal characteristics in both time and frequency domains.

A Complete Figure Documentation

All figures presented in this report were generated from MATLAB simulations processing the audio file 'ahmed.m4a'. The complete set includes:

- Original and noisy signal representations (Figure 1)
- Modulation technique analyses (Figures 2-4)
- Filter characteristics (Figures 5-7)

- Filtered signal results (Figures 9-14)

Each figure is accompanied by detailed technical captions explaining the key features and processing effects visible in the visualizations.

Full MATLAB Code

```
%
    ------------------------------------------------------------------------

% STEP 1: Load the Audio File
%
    ------------------------------------------------------------------------


[audio, fs] = audioread('ahmed.m4a');    % Load audio file and its
    sampling rate
t = (0:length(audio)-1) / fs;            % Generate time vector
   based on sampling frequency

% Check if the audio is stereo (2 channels), and convert to mono
   if needed
if size(audio, 2) == 2
    audio = mean(audio, 2);              % Take the average of
       both channels
end

%
    ------------------------------------------------------------------------

% STEP 2: Frequency Domain Analysis of the Original Audio
%
    ------------------------------------------------------------------------


N = length(audio);                       % Number of samples in
   the audio signal
f = (-N/2:N/2-1) * (fs/N);               % Frequency axis for FFT
   (centered)
audio_fft = fftshift(fft(audio));        % Compute centered FFT
   for spectrum analysis

%
    ------------------------------------------------------------------------

% STEP 3: Add White Gaussian Noise to the Audio
%
    ------------------------------------------------------------------------


SNR = 10;                                % Set signal-to-noise
   ratio in decibels
noisy_audio = awgn(audio, SNR, 'measured');  % Add white noise
   preserving SNR

```

```
%
    ------------------------------------------------------------------------

% STEP 4: Time and Frequency Domain Plots
%
    ------------------------------------------------------------------------


% Time and Frequency domain of original audio
figure;
subplot(2,1,1);
plot(t, audio);
title('Original Audio Signal (Time Domain)');
xlabel('Time (s)'); ylabel('Amplitude');

subplot(2,1,2);
plot(f, abs(audio_fft));
title('Original Audio Signal (Frequency Domain)');
xlabel('Frequency (Hz)'); ylabel('Magnitude');

% Time and Frequency domain of noisy audio
figure;
subplot(2,1,1);
plot(t, noisy_audio);
title(['Noisy Audio Signal (Time Domain) - SNR = ', num2str(SNR),
    ' dB']);
xlabel('Time (s)'); ylabel('Amplitude');

noisy_audio_fft = fftshift(fft(noisy_audio)); % FFT of noisy
    signal
subplot(2,1,2);
plot(f, abs(noisy_audio_fft));
title('Noisy Audio Signal (Frequency Domain)');
xlabel('Frequency (Hz)'); ylabel('Magnitude');

% Ensure vectors are column-aligned
noisy_audio = noisy_audio(:);
t = (0:length(noisy_audio)-1)' / fs;

%
    ------------------------------------------------------------------------

% STEP 5: AM Modulation (Amplitude Modulation)
%
    ------------------------------------------------------------------------


fc_am = 1000;  % Carrier frequency for modulation (Hz)
modulated_am = (1 + 0.5 * noisy_audio) .* cos(2 * pi * fc_am * t)
    ;  % Generate AM signal
```

```
modulated_am_fft = fftshift(fft(modulated_am));  % FFT for spectral analysis

% Plot AM modulated signal
figure;
subplot(2,1,1); plot(t, modulated_am);
title('AM Modulated Signal in Time Domain');
xlabel('Time (s)'); ylabel('Amplitude');

subplot(2,1,2); plot(f, abs(modulated_am_fft));
title('Frequency Domain of AM Modulated Signal');
xlabel('Frequency (Hz)'); ylabel('Magnitude');

% ------------------------------------------------------------------------

% STEP 6: DSB-SC Modulation (Double Sideband - Suppressed Carrier)
% ------------------------------------------------------------------------


fc_dsb = 1000;  % Carrier frequency
modulated_dsb_sc = noisy_audio .* cos(2 * pi * fc_dsb * t); % Multiply with carrier only
modulated_dsb_sc_fft = fftshift(fft(modulated_dsb_sc));     % FFT for frequency analysis

% Plot DSB-SC
figure;
subplot(2,1,1); plot(t, modulated_dsb_sc);
title('DSB-SC Modulated Signal in Time Domain');
xlabel('Time (s)'); ylabel('Amplitude');

subplot(2,1,2); plot(f, abs(modulated_dsb_sc_fft));
title('Frequency Domain of DSB-SC Modulated Signal');
xlabel('Frequency (Hz)'); ylabel('Magnitude');

% ------------------------------------------------------------------------

% STEP 7: DSB-TC Modulation (Double Sideband - Transmitted Carrier)
% ------------------------------------------------------------------------


fc_tc = 1000;  % Carrier frequency
modulated_dsb_tc = (1 + noisy_audio) .* cos(2 * pi * fc_tc * t);
    % Add carrier
```

```
103 modulated_dsb_tc_fft = fftshift(fft(modulated_dsb_tc));
    % FFT
104
105 % Plot DSB-TC
106 figure;
107 subplot(2,1,1); plot(t, modulated_dsb_tc);
108 title('DSB-TC Modulated Signal in Time Domain');
109 xlabel('Time (s)'); ylabel('Amplitude');
110
111 subplot(2,1,2); plot(f, abs(modulated_dsb_tc_fft));
112 title('Frequency Domain of DSB-TC Modulated Signal');
113 xlabel('Frequency (Hz)'); ylabel('Magnitude');
114
115 %
    ------------------------------------------------------------------------

116 % STEP 8: FM Modulation (Frequency Modulation)
117 %
    ------------------------------------------------------------------------

118
119 fc_fm = 1000;                          % Carrier frequency
120 freq_dev = 500;                        % Frequency deviation
121 % Integral of the signal used in FM phase expression
122 modulated_fm = cos(2 * pi * fc_fm * t + 2 * pi * freq_dev *
    cumsum(noisy_audio) / fs);
123 modulated_fm_fft = fftshift(fft(modulated_fm));
124
125 % Plot FM signal
126 figure;
127 subplot(2,1,1); plot(t, modulated_fm);
128 title('FM Modulated Signal in Time Domain');
129 xlabel('Time (s)'); ylabel('Amplitude');
130
131 subplot(2,1,2); plot(f, abs(modulated_fm_fft));
132 title('Frequency Domain of FM Modulated Signal');
133 xlabel('Frequency (Hz)'); ylabel('Magnitude');
134
135 %
    ------------------------------------------------------------------------

136 % STEP 9: 100 Hz Low-Pass Butterworth Filter Design
137 %
    ------------------------------------------------------------------------

138
139 fc = 100;                              % Cutoff frequency (Hz)
140 Wn = fc / (fs/2);                      % Normalize cutoff by
    Nyquist
141 [b, a] = butter(4, Wn);                % 4th-order Butterworth
    design
```

```
142
143 % Plot pole-zero diagram
144 figure; zplane(b, a);
145 title('Pole-Zero Plot of 100 Hz Low-Pass Filter');
146
147 % Calculate region of convergence (ROC)
148 poles = roots(a);
149 roc_radius = max(abs(poles));
150 disp(['ROC: |z| > ', num2str(roc_radius)]);
151
152 % Shade ROC region in z-plane
153 theta = linspace(0, 2*pi, 500);
154 radii = linspace(roc_radius, 1.5, 300);
155 figure; hold on;
156 for r = radii
157     plot(r*cos(theta), r*sin(theta), 'Color', [0.9 0.9 1], '
        LineWidth', 0.5);
158 end
159 z = roots(b); % zeros
160 plot(real(z), imag(z), 'bo', 'LineWidth', 1.5);      % Plot zeros
161 plot(real(poles), imag(poles), 'rx', 'LineWidth', 1.5); % Plot
    poles
162 plot(cos(theta), sin(theta), 'k--');                 % Unit
    circle
163 title('Z-Plane with Region of Convergence (ROC)');
164 legend('ROC', 'Zeros', 'Poles', 'Unit Circle'); axis equal; grid
    on;
165
166 [H, f_response] = freqz(b, a, 1024, fs);
167
168 figure('Color','w');
169 tiledlayout(2,1, 'Padding', 'compact', 'TileSpacing', 'compact');
170
171 % Magnitude
172 nexttile;
173 plot(f_response, 20*log10(abs(H)), 'b', 'LineWidth', 1.2);
174 grid on; box on;
175 title('100 Hz Low-Pass Filter - Magnitude Response', 'FontWeight'
    , 'bold');
176 xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
177 set(gca, 'FontName', 'Arial');
178
179 % Phase
180 nexttile;
181 plot(f_response, unwrap(angle(H)) * 180/pi, 'r', 'LineWidth',
    1.2);
182 grid on; box on;
183 title('100 Hz Low-Pass Filter - Phase Response', 'FontWeight', '
    bold');
184 xlabel('Frequency (Hz)'); ylabel('Phase (degrees)');
185 set(gca, 'FontName', 'Arial');
```

```
186
187
188 % Apply LPF to noisy signal
189 filtered_audio = filter(b, a, noisy_audio);
190
191 % Plot LPF output
192 figure;
193 subplot(2,1,1); plot(t, filtered_audio);
194 title('Filtered Signal (100 Hz LPF) - Time Domain');
195
196 subplot(2,1,2);
197 filtered_fft = fftshift(fft(filtered_audio));
198 plot(f, abs(filtered_fft));
199 title('Filtered Signal (100 Hz LPF) - Frequency Domain');
200
201 %
    ------------------------------------------------------------------------

202 % STEP 10: Reapply Modulations After 100 Hz LPF
203 %
    ------------------------------------------------------------------------

204
205 % AM
206 mod_am = (1 + 0.5 * filtered_audio) .* cos(2*pi*fc_am*t);
207 mod_am_fft = fftshift(fft(mod_am, N));
208 figure;
209 subplot(2,1,1); plot(t, mod_am); title('AM Modulated (100 Hz LPF)
     - Time');
210 subplot(2,1,2); plot(f, abs(mod_am_fft)); title('AM Modulated
    (100 Hz LPF) - Freq');
211
212 % DSB-SC
213 mod_dsb = filtered_audio .* cos(2*pi*fc_am*t);
214 mod_dsb_fft = fftshift(fft(mod_dsb, N));
215 figure;
216 subplot(2,1,1); plot(t, mod_dsb); title('DSB-SC Modulated (100 Hz
     LPF) - Time');
217 subplot(2,1,2); plot(f, abs(mod_dsb_fft)); title('DSB-SC
    Modulated (100 Hz LPF) - Freq');
218
219 % DSB-TC
220 mod_tc = (1 + filtered_audio) .* cos(2*pi*fc_am*t);
221 mod_tc_fft = fftshift(fft(mod_tc, N));
222 figure;
223 subplot(2,1,1); plot(t, mod_tc); title('DSB-TC Modulated (100 Hz
    LPF) - Time');
224 subplot(2,1,2); plot(f, abs(mod_tc_fft)); title('DSB-TC Modulated
     (100 Hz LPF) - Freq');
225
226 % FM
```

```
mod_fm = cos(2*pi*fc_am*t + 2*pi*freq_dev*cumsum(filtered_audio)/fs);
mod_fm_fft = fftshift(fft(mod_fm, N));
figure;
subplot(2,1,1); plot(t, mod_fm); title('FM Modulated (100 Hz LPF) - Time');
subplot(2,1,2); plot(f, abs(mod_fm_fft)); title('FM Modulated (100 Hz LPF) - Freq');

%% ------------------------------------------------------------------------

% STEP 11: Design a 60 Hz Notch Filter
%% ------------------------------------------------------------------------


f_notch = 60;                          % Target frequency to remove
r = 0.95;                              % Radius (close to 1 = narrow notch)
omega = 2*pi*f_notch/fs;              % Normalized angular frequency

% Coefficients for notch filter (second-order IIR)
b = [1, -2*cos(omega), 1];
a = [1, -2*r*cos(omega), r^2];

% Pole-zero plot
figure; zplane(b, a);
title('60 Hz Notch Filter - Pole-Zero Plot');

% Region of Convergence (ROC) for stability
poles = roots(a);
roc_radius = max(abs(poles));
disp(['ROC: |z| > ', num2str(roc_radius)]);

% Plot ROC
theta = linspace(0, 2*pi, 500);
radii = linspace(roc_radius, 1.5, 300);
figure; hold on;
for r_shade = radii
    plot(r_shade*cos(theta), r_shade*sin(theta), 'Color', [1, 0.9, 0.9], 'LineWidth', 0.5);
end
z = roots(b);
plot(real(z), imag(z), 'bo', 'LineWidth', 1.5);         % Zeros
plot(real(poles), imag(poles), 'rx', 'LineWidth', 1.5);% Poles
plot(cos(theta), sin(theta), 'k--');                    % Unit circle
title('Z-Plane with ROC - 60 Hz Notch Filter'); axis equal; grid on;

```

```
[H_notch, f_notch_response] = freqz(b, a, 1024, fs);

figure('Color','w');
tiledlayout(2,1, 'Padding', 'compact', 'TileSpacing', 'compact');

% Magnitude
nexttile;
plot(f_notch_response, 20*log10(abs(H_notch)), 'b', 'LineWidth', 1.2);
grid on; box on;
title('60 Hz Notch Filter - Magnitude Response', 'FontWeight', 'bold');
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
set(gca, 'FontName', 'Arial');

% Phase
nexttile;
plot(f_notch_response, unwrap(angle(H_notch)) * 180/pi, 'r', 'LineWidth', 1.2);
grid on; box on;
title('60 Hz Notch Filter - Phase Response', 'FontWeight', 'bold');
xlabel('Frequency (Hz)'); ylabel('Phase (degrees)');
set(gca, 'FontName', 'Arial');


% Apply notch filter
filtered_audio = filter(b, a, noisy_audio);

% Time and Frequency plots after notch filter
figure;
subplot(2,1,1); plot(t, filtered_audio);
title('Filtered Signal (60 Hz Notch) - Time Domain');

subplot(2,1,2);
filtered_fft = fftshift(fft(filtered_audio));
plot(f, abs(filtered_fft));
title('Filtered Signal (60 Hz Notch) - Frequency Domain');

%% ------------------------------------------------------------------------

% STEP 12: Apply Modulations After 60 Hz Notch Filtering
%% ------------------------------------------------------------------------


% AM
mod_am = (1 + 0.5 * filtered_audio) .* cos(2*pi*fc_am*t);
mod_am_fft = fftshift(fft(mod_am));
figure;
```

```
subplot(2,1,1); plot(t, mod_am); title('AM Modulated (60 Hz Notch)');
subplot(2,1,2); plot(f, abs(mod_am_fft)); title('AM Spectrum (60 Hz Notch)');

% DSB-SC
mod_dsb = filtered_audio .* cos(2*pi*fc_am*t);
mod_dsb_fft = fftshift(fft(mod_dsb));
figure;
subplot(2,1,1); plot(t, mod_dsb); title('DSB-SC Modulated (60 Hz Notch)');
subplot(2,1,2); plot(f, abs(mod_dsb_fft)); title('DSB-SC Spectrum (60 Hz Notch)');

% DSB-TC
mod_tc = (1 + filtered_audio) .* cos(2*pi*fc_am*t);
mod_tc_fft = fftshift(fft(mod_tc));
figure;
subplot(2,1,1); plot(t, mod_tc); title('DSB-TC Modulated (60 Hz Notch)');
subplot(2,1,2); plot(f, abs(mod_tc_fft)); title('DSB-TC Spectrum (60 Hz Notch)');

% FM
mod_fm = cos(2*pi*fc_am*t + 2*pi*freq_dev*cumsum(filtered_audio)/fs);
mod_fm_fft = fftshift(fft(mod_fm));
figure;
subplot(2,1,1); plot(t, mod_fm); title('FM Modulated (60 Hz Notch)');
subplot(2,1,2); plot(f, abs(mod_fm_fft)); title('FM Spectrum (60 Hz Notch)');
```

Listing 3: Full MATLAB Implementation